

Estado del Proyecto

Simulador de Pedidos de Cafetería · TPI Programación Avanzada 2026

Requisitos del TPI — estado

Requisito	Estado
Aplicar POO (4 pilares)	LISTO
Al menos un patrón de diseño (Decorator)	LISTO
3 o 4 clases mínimo (hay 5)	LISTO
Una relación entre objetos (agregación)	LISTO
Funcional y demostrable en consola	LISTO
README con instrucciones de ejecución	LISTO (falta poner integrantes)
Diagrama de clases UML	PENDIENTE
Diapositivas de la exposición (8-15)	PENDIENTE
Repositorio en GitHub/GitLab	A CREAR

Los requisitos obligatorios del TPI están cumplidos. Base de datos, API REST, Docker y frameworks son opcionales y no se incluyen, para mantener el alcance acotado.

Qué hace el sistema

Aplicación de consola: herramienta interna del mozo y el gerente (el cliente no la usa). El mozo inicia sesión, arma pedidos eligiendo del menú (por categorías, con precio y stock), agrega extras a las bebidas, ve el total con propina del 10% y emite el ticket. El gerente administra el menú (precios, stock) y consulta el cierre del día consolidado.

Conceptos aplicados (para la defensa oral)

- **Abstracción:** Producto es clase abstracta (ABC) con método abstracto.
- **Herencia:** Bebida y Comida heredan de Producto.
- **Polimorfismo:** cada producto y cada Extra resuelve precio y descripción a su manera.
- **Encapsulamiento:** clave hashada (SHA-256) en atributo privado; precio de solo lectura.
- **Relación:** agregación — la Cafetería agrupa Mozos; cada Mozo agrupa sus Pedidos.
- **Patrón Decorator (GoF):** clase Extra que envuelve un Producto (extras del café).
- **Decoradores de función (@):** @verificar_stock y @requiere_login — herramienta distinta del patrón.

Decisiones de diseño (por qué se hizo así)

- **¿Por qué Decorator y no subclases?** Para no crear una clase por cada combinación (CaféConLecheGrande...). Los extras se combinan en ejecución.
- **¿Por qué categoría como Enum y no subclase?** Un producto pertenece a una sola categoría; es un atributo, no una jerarquía.
- **¿Por qué "sin TACC"/"vegetariano" como banderas?** Son independientes y se cruzan entre sí y con la categoría; como subclases explotarían en combinaciones.
- **¿Por qué modularizado en varios archivos?** Para que cada parte tenga una responsabilidad clara y sea fácil de mantener, corregir y escalar. La consola no sabe de negocio y el negocio no sabe de consola.

Cómo ejecutarlo

Requiere Python 3.10 o superior, sin librerías externas. Pararse en la carpeta raíz del proyecto (la que contiene `main.py`) y ejecutar: `python main.py`. Mozos de prueba: Carlos, María, José (clave 1234). Importante: ejecutar desde la carpeta raíz, no desde una subcarpeta, para que funcionen los imports.

Próximos pasos

- 1 Descargar el .zip y armar la carpeta del proyecto.
- 2 Crear el repositorio en GitHub y subir el contenido.
- 3 Completar la sección "Integrantes" del README con los nombres del grupo.
- 4 Armar el diagrama de clases UML (próximo paso recomendado).
- 5 Preparar las diapositivas (8 a 15) para la exposición.
- 6 Practicar la demostración en vivo y las respuestas de la defensa.